Final Project: Group 55

Udritty Das, Scott Reader, Matthew Woodcock

McMaster University

COMPSCI 2XC3: Computer Science Practice and Experience: Algorithms and Software Design

Dr. Swati Mishra

April 9, 2025

Table of Contents:

Team Charter

1. Communication Protocol:

- We will communicate via Instagram messages and will expect a response within 12 hours of a message being sent if urgent, 48 otherwise. Members are expected to check the chat daily.

2. Communication Agreement Penalty:

- If a team member repeatedly fails to adhere to the communication agreement, the entire team will have a meeting to discuss expectations, redetermine goals and work distribution if necessary, and continue work.

3. Collaboration Technology and Version Control:

- **Collaboration Tools:** Code will be assigned per question for work and will be completed through VS code.
- **Version Control:** Completed work will be submitted to a shared repository. Work that is not completed or that requires assistance will also be put there with respective push messages.

4. Dispute Resolution:

- Team members will attempt to resolve disputes by scheduling a dedicated meeting to openly discuss the issue, listen to each other's perspectives, and work towards a mutually agreeable solution.
- If an agreement is not reached within two meetings, we will escalate the matter to the TA for mediation.

6. Signatures:

Name	Student #	Student ID
Scott Reader	400339095	readers
Matthew Woodcock	400613327	woodcocm
Uddrity Das	400364113	dasu

Part 1

Single Source Shortest Path Algorithms

See Dijkstra_BellmanFord.py for relaxed algorithm implementations.

Experiment:

Our experiment compares the performance of these two algorithms on a random graph with a predetermined number of nodes and graphs. Large and small graphs were tested, dense and sparse of each were also considered, and the k value was varied, in the table of charts in the table of figures at the end of the report.

A higher k value consistently resulted in slower performance for Bellman-Ford, although did not impact Dijkstra nearly as much. When the k value was set lower Bellman-Ford outperformed Dijkstra marginally. This trend was seen for both dense and sparse graphs, and the figures representing this can be seen at the end of the report.

Part 2

All-Pair Shortest Path Algorithm

For implementation see Johnson.py.

Our approach was to implement Johnson's algorithm, as it takes advantage of Bellman-Ford and Dijkstra's algorithms that were already implemented.

As for the complexity of Dijkstra and Bellman-Ford, for a dense graph the relaxed version of these algorithms will have different complexities than the traditional ones. When looking at Dijkstra, the complexity of the relaxed version would be $O(k \cdot V^2 \cdot \log V)$ for a dense graph, as the number of edges is restricted by the limit set by k, resulting in it performing similar to standard Dijkstra on an average graph. Similar logic can be applied to the relaxed Bellman-Ford algorithm, where it would have a complexity of $O(k \cdot V^2)$, as the number of edges is restricted again. Like Dijkstra, this means that the relaxed version will run similar for a dense graph to how standard Bellman-Ford will perform for an average graph.

The performance of the relaxed algorithm is also reflected in the performance of Johnson's algorithm, as it makes use of the relaxed versions of these two algorithms.

Part 3

A* algorithm

For implementation see A_Star.py.

A* tries to address the blind selection of Dijkstra by weighing the cost of each movement it makes before selecting, a downside for Dijkstra. Dijkstra is also very slow for large or dense graphs, whereas A* performs much better. Dijkstra vs A* would be best tested against each other by comparing them on a practical real-world dataset. This experiment would best be done using a real-world graph, such as the London subway provided for this report. The test cases would involve comparing the performance speed and the number of lines, the edges, that they use to reach the same destination node, or station. Various heuristics could be tested, such as Euclidean, Manhattan, or diagonal distance. These could then be plotted and compared visually. This could be repeated several times to ensure that runtime fluctuations are accounted for.

Using an arbitrary heuristic function would result in A* likely performing worse when compared to Dijkstra. This is because the heuristic would likely not be admissible, and would potentially overestimate the cost, resulting in unnecessary exploration and not guaranteeing the shortest path.

Knowing all of this, A* is best suited for situations involving a situation that has a good heuristic, such as Euclidean distance, rather than using Dijkstra. Similarly, for faster pathfinding, or for large graphs A* would provide faster response times and would be better suited. Inversely, when there is no good heuristic available, Dijkstra will outperform A*.

Compare Shortest Path Algorithms

For implementation see London_Subway.py, London_Dikjstra.py and A_Star.py.

Experiment:

A* generally appears to outperform Dijkstra, but in some instances they do perform similarly. A* did perform worst using the diagonal distance heuristic and performed comparably for the Manhattan and Euclidean distance heuristics. These trends can be seen in the figures at the end of the report

When stations are on the same lines, A* outperforms Dijkstra consistently, the former taking less than half the time of the latter consistently. When stations are on adjacent lines the performance is similar to when there is no line change. When stations are on lines that require several transfers A* does begin to perform similarly to Dijkstra much of the time.

Part 4

UML Diagram Discussion

This UML diagram follows the SOLID design principles very well. Each class is very clearly responsible for a single goal, they do not allow for modification but are clearly extendable, they don't violate LSP, each class is contained and not overreaching, and the high-level components are abstract, while the low levels ones are not.

The design patterns present in the UML diagram include template, strategy, decorator, and adapter. Strategy is seen in ShortPathFinder, as it can use any of the algorithms through the interface that is implemented. Template is present in the SPAlgorithm interface would likely have some base class that each of the algorithms overwrite to employ their respective approach. Decorator is seen in WeightedGraph and HeuristicGraph, as the former extends the base Graph interface by adding different functionality, and the latter does the same to WeightedGraph. Adapter is obviously seen in the implementation of A_Star.

To make the design in the figure more robust, the nodes could be altered to be a generic node type so that more than just integers can be accepted. Further modifying the nodes, a node class or interface could be implemented to better handle additional information in a node. By abstracting the node, their types could be checked at compile time. And would allow for a greater variety of nodes, both simple and complex. This also prevents the entire code structure to be redone to handle variable node types.

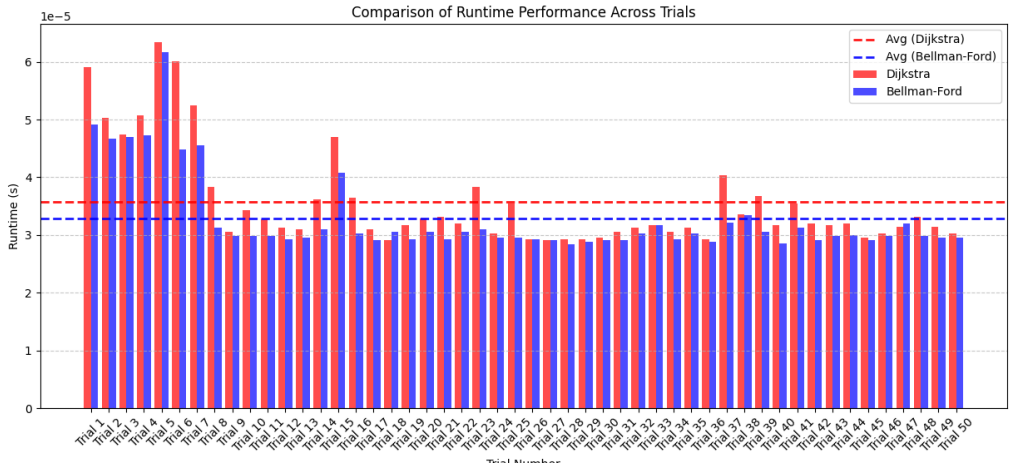
Summary

When comparing Dijkstra and Bellman-Ford, the performance of the latter was noticeably worse when k was higher for both dense and sparse graphs. When k was lower, it consistently performed slightly better than Dijkstra. Both performed noticeably slower when the graphs were dense.

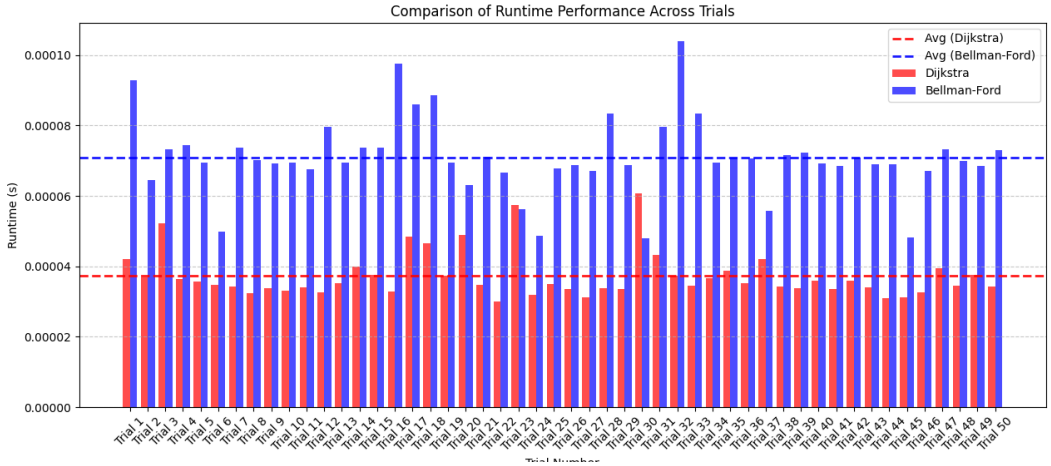
When comparing A* and Dijkstra on the London Subway dataset, A* was consistently faster when using all 3 heuristics, though did perform more similar to Dijkstra when using the diagonal heuristic. This reflects the situations when A* was discussed to be better suited to the task then Dijkstra, as they were tested on a real-world dataset.

Table of Figures:

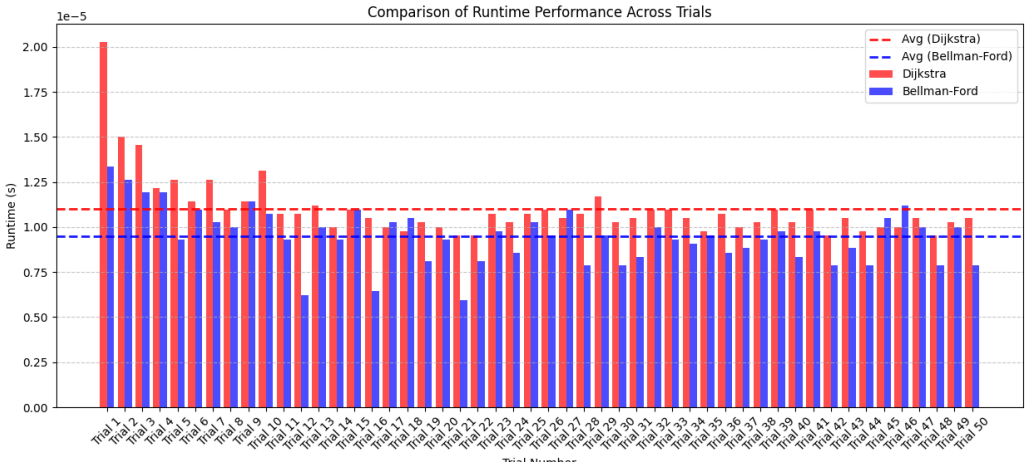
Dense (n=10, e=45), k=2:



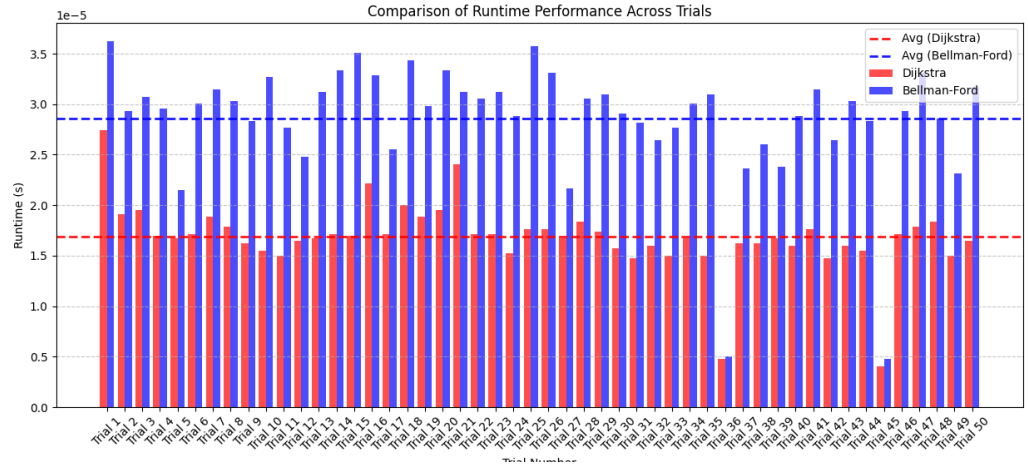
Dense (n=10, e=45), k=4:



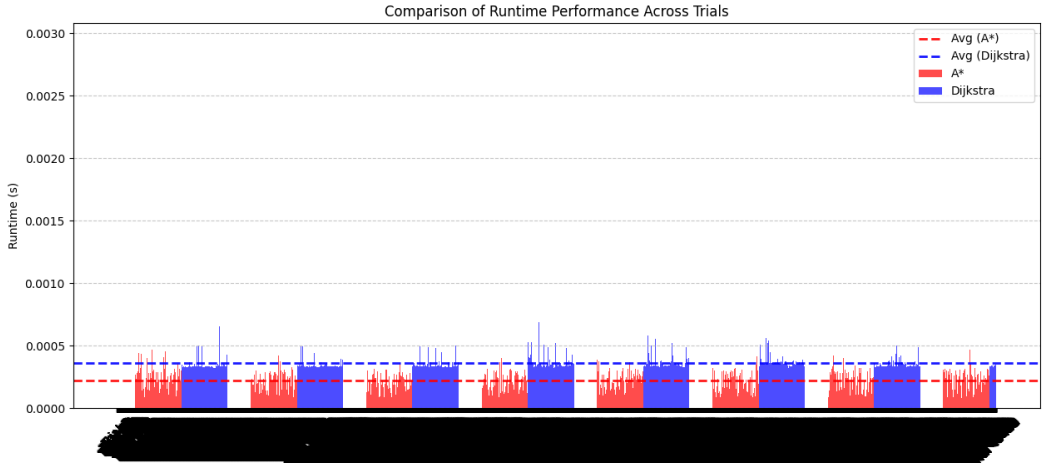
Sparse (n=7, e=10), k=2:



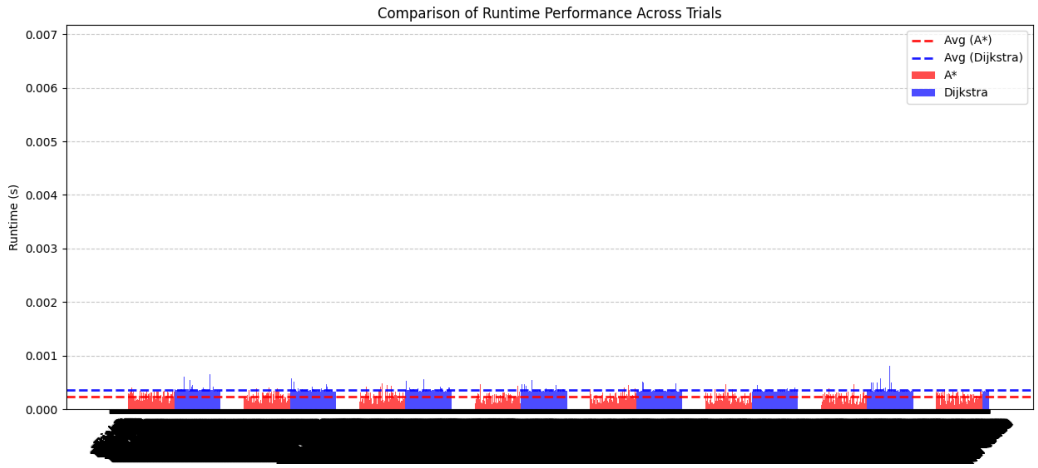
Sparse (n=7, e=10) k=4:



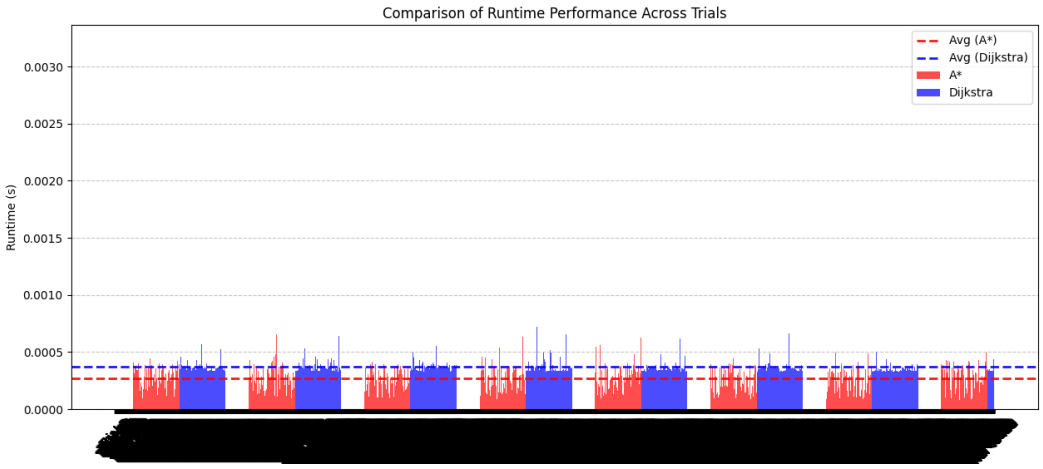
A* Manhattan vs Dijkstra:



A* Diagonal vs Dijkstra:



A* Euclidean vs Dijkstra:



Appendix

How to navigate the code:

The code is split across 10 files, the names indicative of the tasks from each section. To test the experiment for part 2, simply run Dijkstra_BellmanFord.py. To test the experiment from part 5, run the London_Subway.py after manually changing the heuristic function on line 100, and the k value can be set for Dijkstra, but can also be set as None on line 106. Both of those are within the compare_algorithms() function. A separate Dijkstra is used for part 5 rather than using the one from part 2, and is titled London_Dijkstra,py. The code may buffer when running for London_Subway.py due to the large dataset being formatted using matplotlib. The files a_star_adapter.py, experiment.py, graph.py, and shortest_paths.py are to align our code with the structure outlined by the UML diagram from part 6.